

Shopizer E-Commerce Platform

Part 5. Testable Design and Mocking

Course: Software Testing

Student: Yijun Sun, Yu Chien Chiu

Date: March 2, 2026

Assignment 5: Testable Design and Mocking

Part 1: Testable Design

1.1 Aspects and Goals of Testable Design

A testable design is one that allows individual components to be verified in isolation from their environment. The primary goals include:

- Isolation (Decoupling): Components should not depend on concrete implementations of external systems (databases, file systems, network). Instead, they should depend on abstractions (interfaces).
- Controllability: The tester should be able to force the software into specific states or paths (e.g., simulating a network timeout or a specific database return value).
- Determinism: A testable design ensures that given the same input and environment state, the component produces the same output every time.
- Observability: The internal state or outputs of a component should be easily accessible for verification.

1.2 Stubbing Implementation

Existing Use of Stubbing: In the refactored `IntegrationModulesLoader`, we introduced the `IntegrationModuleSource` interface.

- Usage: The system now uses this interface to load raw data instead of directly accessing the classpath resources.
- Rationale: This is used because the original implementation was tightly coupled to the Java `ClassLoader` and resource files. By stubbing this source, we can return pre-defined JSON strings in memory, making the tests fast and independent of the physical disk.

```
private class DefaultIntegrationModuleSource implements IntegrationModuleSource {
    @Override
    public String loadRawJson(String identifier) throws ServiceException {
        try {
            InputStream in = IntegrationModulesLoader.this.getClass().getClassLoader()
                .getResourceAsStream(identifier);
            if (in == null) {
                throw new ServiceException("File not found: " + identifier);
            }
            java.util.Scanner s = new java.util.Scanner(in).useDelimiter("\\A");
            return s.hasNext() ? s.next() : "";
        } catch (Exception e) {
            throw new ServiceException(e);
        }
    }
}
```

New Stub Implementation: I implemented `StubIntegrationModuleSource` inside `IntegrationModulesLoaderTest.java`.

- Technique: I created a static inner class that implements the IntegrationModuleSource interface.
- Test Case: In testLoadModulesWithValidJson, this stub is injected into the loader. It returns a hardcoded JSON string, allowing us to verify the parser's logic without ever reading a real file.

```

3  @Test
4  public void testLoadModulesWithValidJson() throws Exception {
5      String validJson = "[{"module": "payment", "code": "paypal", "image": "paypal.png"}]";
6
7      IntegrationModulesLoader.IntegrationModuleSource stubSource = new StubIntegrationModuleSource(validJson);
8      IntegrationModulesLoader.IntegrationModuleParser mockParser = mock(
9          IntegrationModulesLoader.IntegrationModuleParser.class);
10
11      List<IntegrationModule> expectedModules = new ArrayList<>();
12      IntegrationModule module = new IntegrationModule();
13      module.setCode("paypal");
14      expectedModules.add(module);
15
16      when(mockParser.parseModules(validJson)).thenReturn(expectedModules);
17
18      IntegrationModulesLoader loader = new IntegrationModulesLoader(stubSource, mockParser);
19      List<IntegrationModule> result = loader.loadIntegrationModules("data/modules.json");
20
21      assertNotNull(result);
22      assertEquals(1, result.size());
23      assertEquals("paypal", result.get(0).getCode());
24      verify(mockParser).parseModules(validJson);
25  }
26
27  @Test
28  public void testLoadModulesWithSourceException() throws Exception {
29      StubIntegrationModuleSource stubSource = new StubIntegrationModuleSource("");
30      stubSource.setThrowException(true);
31      stubSource.setExceptionMessage("file not found");
32
33      IntegrationModulesLoader.IntegrationModuleParser mockParser = mock(
34          IntegrationModulesLoader.IntegrationModuleParser.class);
35      IntegrationModulesLoader loader = new IntegrationModulesLoader(stubSource, mockParser);
36
37      assertThrows(ServiceException.class, () -> {
38          loader.loadIntegrationModules("nonexistent.json");
39      });
40  }

```

1.3 Analysis of Bad Testable Design part1

Documented Code (Original Design)

The original IntegrationModulesLoader.loadIntegrationModules method contained the following logic:

```

InputStream in =
this.getClass().getClassLoader().getResourceAsStream(jsonFilePath);
Map[] objects = mapper.readValue(in, Map[].class);

```

The Problem

This design makes testing difficult because:

1. File System Dependency: You cannot test how the code handles different JSON structures without creating multiple physical files in the `src/main/resources` folder.
2. Error Handling: It is nearly impossible to simulate an `InputStream` failure or a specific I/O error without complex environment manipulation.
3. Side Effects: Tests might accidentally overwrite or depend on shared resource files, leading to flaky tests.

Advice for Fix

Replace the hardcoded instantiation/access with Dependency Injection (DI). Define an interface for the resource loading behavior and pass that interface into the constructor.

Implementation of New Design

I updated IntegrationModulesLoader to accept two interfaces:

- IntegrationModuleSource (for getting the string)
- IntegrationModuleParser (for converting data to objects)

```
private class DefaultIntegrationModuleSource implements IntegrationModuleSource {
    @Override
    public String loadRawJson(String identifier) throws ServiceException {
        try {
            InputStream in = IntegrationModulesLoader.this.getClass().getClassLoader()
                .getResourceAsStream(identifier);
            if (in == null) {
                throw new ServiceException("File not found: " + identifier);
            }
            java.util.Scanner s = new java.util.Scanner(in).useDelimiter("\\A");
            return s.hasNext() ? s.next() : "";
        } catch (Exception e) {
            throw new ServiceException(e);
        }
    }
}

private class DefaultIntegrationModuleParser implements IntegrationModuleParser {
    @Override
    public List<IntegrationModule> parseModules(String jsonContent) throws ServiceException {
        List<IntegrationModule> modules = new ArrayList<IntegrationModule>();
        ObjectMapper mapper = new ObjectMapper();
        try {
            @SuppressWarnings("rawtypes")
            Map[] objects = mapper.readValue(jsonContent, Map[].class);
            for (Map object : objects) {
                modules.add(loadModule(object));
            }
            return modules;
        } catch (Exception e) {
            throw new ServiceException(e);
        }
    }
}
```

1.4 Analysis of Bad Testable Design part2 (Yu-Chien Chiu)

1. Problematic Design in Original Implementation

In the original implementation of `ShoppingCartServiceImpl.saveOrUpdate()`, the system directly retrieves user context using a static global call:

[Problem code for bad design](#)

Why This Is Bad Design for Testing:

This design tightly couples business logic to a static global dependency.

`UserContext.getCurrentInstance()`:

- Is a static method
- Returns a global object
- Belongs to a final class
- Cannot be easily mocked with standard Mockito

What Testing Becomes Difficult or Impossible?

- Cannot control what `UserContext.getCurrentInstance()` returns
- Cannot simulate "no user context" reliably
- Cannot simulate different IP addresses
- Cannot verify interaction behavior
- Cannot isolate cart logic from global application state

This violates:

- Dependency Inversion Principle
- Single Responsibility Principle
- Unit Test Isolation principles

The `saveOrUpdate()` method becomes dependent on global state, making proper unit testing difficult.

2. Recommended Fix

To improve testability, we refactored the design to introduce an abstraction layer. Instead of calling `UserContext.getCurrentInstance()` directly, we created an interface: `UserContextProvider` [UserContextProvider](#)

3. Refactored Version ('ShoppingCartUserContextV2')

We implemented a new version that depends on the abstraction: [FixBadDesign](#)

Now the business logic no longer depends on static global state.

4. Why This Fix Improves Testability

Now:

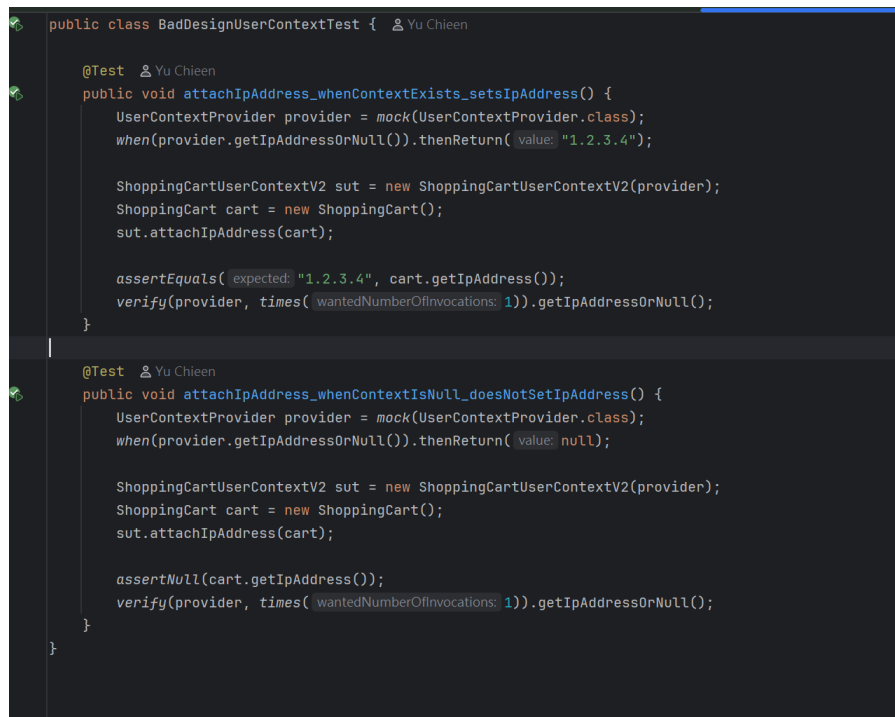
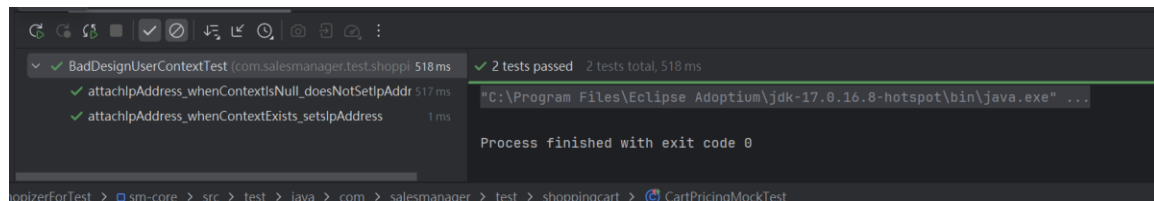
- We can inject a mock implementation
- We can simulate different IP addresses
- We can simulate null context
- No static calls exist inside business logic
- Behavior can be verified independently

This follows:

- Dependency Injection
- SOLID principles
- Clean Architecture principles
- Testable design practices

5. New Test Case for More Testable Code

We implemented a unit test:



What This Test Achieves:

- Confirms IP is attached when context exists
- Simulates absence of context
- Does not depend on global application state
- Fully unit-testable

This functionality was previously difficult to test due to static global dependency.

Part 2: Mocking

2.1 Mocking and its Utility

Mocking involves creating objects that simulate the behavior of real, complex objects. Its primary utilities are:

- **Verification of Interactions:** Unlike stubs (which just provide data), mocks allow you to verify if a method was called, with what arguments, and how many times.
- **Avoiding Side Effects:** Mocking an EmailService ensures you don't actually send 1,000 emails during a test run.
- **Simulating External Systems:** Mocks can easily simulate external API failures or specific sequence of calls that are hard to trigger with real systems.

Extended Theory

What is Mocking?

Mocking is a unit testing technique in which real dependencies are replaced with controlled test doubles (mock objects). Mocking allows the tester to isolate the system under test (SUT) from its external dependencies.

Mocks provide:

- **Controllability** – the ability to define exactly what a dependency should return.
- **Behavior Verification** – the ability to verify whether a dependency method was invoked, how many times it was called, and with what arguments.
- **Isolation** – preventing external systems (e.g., APIs, databases, or complex services) from affecting test results.

In this project, Mockito was used to create mock objects and verify behavior.

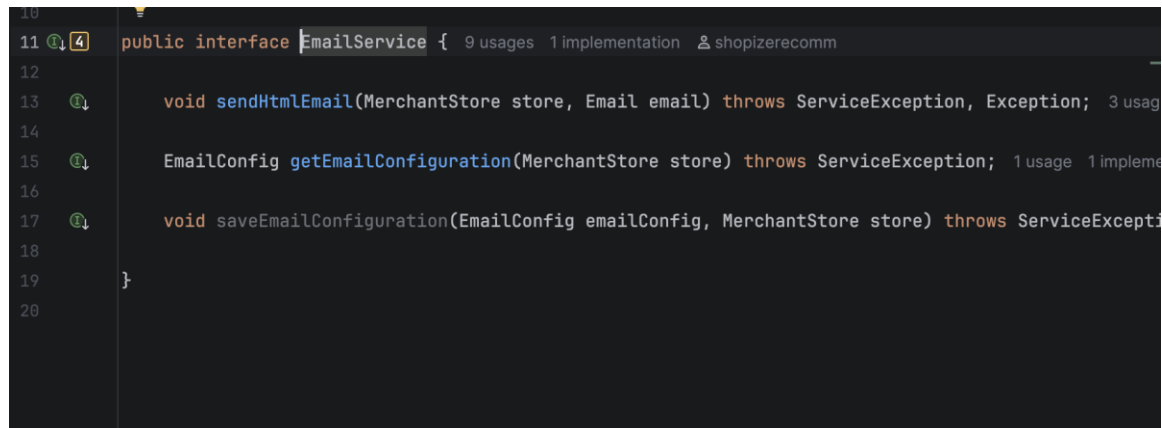
2.2 Feature for Mocking: Email Notification

The EmailService is a perfect candidate for mocking. In most enterprise applications, sending an email involves an SMTP server and network connectivity.

- Behavior Checking: Without mocking, we could only check if the code runs without crashing. With Mockito mocking, we can check if the correct recipient was set and if the Email Template Tokens (like order ID and amount) were populated correctly.

1. Problematic Design in Original Implementation

In the original implementation of `ShoppingCartServiceImpl.saveOrUpdate()`, the system directly retrieves user context using a static global call:



```

10
11 public interface EmailService { 9 usages 1 implementation 2 shopizerecomm
12
13     void sendHtmlEmail(MerchantStore store, Email email) throws ServiceException, Exception; 3 usag
14
15     EmailConfig getEmailConfiguration(MerchantStore store) throws ServiceException; 1 usage 1 impleme
16
17     void saveEmailConfiguration(EmailConfig emailConfig, MerchantStore store) throws ServiceExcepti
18
19 }
20
  
```

Why This Is Bad Design for Testing:

This design tightly couples business logic to a static global dependency. `UserContext.getCurrentInstance()`:

- Is a static method
- Returns a global object
- Belongs to a final class
- Cannot be easily mocked with standard Mockito

What Testing Becomes Difficult or Impossible?

- Cannot control what `UserContext.getCurrentInstance()` returns
- Cannot simulate "no user context" reliably
- Cannot simulate different IP addresses
- Cannot verify interaction behavior
- Cannot isolate cart logic from global application state

2.3 Mockito Test Case(Yijun Sun)

I implemented `EmailServiceMockTest.java` using Mockito.

- Key Technique: Used `@Mock` for `EmailService` and `ArgumentCaptor` to inspect the `Email` object passed to the service.
- Verification: The test `testOrderConfirmationEmailContentIsCorrect` captures the `Email` object and asserts that the subject and template tokens match the test data.


```

@Test  KrimsonSun
public void testOrderConfirmationEmailContentIsCorrect() throws Exception {
    String customerEmail = "john@example.com";
    String orderId = "ORD-67890";
    double totalAmount = 299.50;

    ArgumentCaptor<Email> emailCaptor = ArgumentCaptor.forClass(Email.class);

    notificationService.sendOrderConfirmation(customerEmail, orderId, totalAmount, mockStore);

    verify(mockEmailService).sendHtmlEmail(any(MerchantStore.class), emailCaptor.capture());

    Email email = emailCaptor.getValue();
    assertEquals(customerEmail, email.getTo());
    assertTrue(email.getSubject().contains("ORD-67890"));
    assertEquals( expected: "ORD-67890", email.getTemplateTokens().get("orderId"));
}

@Test  KrimsonSun

```

The screenshot shows an IDE interface with the 'Test Results' tab selected. The left pane displays the following test output:

```

%TESTS 6, testOrderConfirmationThrowsExceptionForEmptyEmail(com.salesma
nager.test.business.services.EmailServiceMockTest)
%TESTE 6, testOrderConfirmationThrowsExceptionForEmptyEmail(com.salesma
nager.test.business.services.EmailServiceMockTest)
%TESTS 7, testAdminNotificationNotSentForNormalOrder(com.salesmanager.t
est.business.services.EmailServiceMockTest)
%TESTE 7, testAdminNotificationNotSentForNormalOrder(com.salesmanager.t
est.business.services.EmailServiceMockTest)
%RUNTIME197

```

The right pane shows the 'Test Runner for Java' window with a list of tests:

- ✓ EmailServiceMockTest \$(sym
- ✓ testAdminNotificationNotSen
- ✓ testAdminNotificationSentFor
- ✓ testOrderConfirmationEmailC
- ✓ testOrderConfirmationEmails
- ✓ testOrderConfirmationHandle
- ✓ testOrderConfirmationThrow.

A link to '3 older results' is visible at the bottom of the test runner window.

2.4 Feature for Mocking: Cart Pricing(Yu Chien Chiu)

The feature selected for mocking is the price calculation logic used by the shopping cart.

In a real system, the PricingService may depend on complex business rules, tax engines, currency configurations, or external pricing logic. Testing such dependencies directly would make unit tests unstable and environment-dependent.

To isolate the cart's behavior, a new testable service (ShoppingCartPricingServiceV2) was introduced. This class depends on PricingService via dependency injection.

Instead of directly instantiating the real PricingService, a mock object is injected into the V2 service to control and verify behavior.

Mocking Implementation

In the test (CartPricingMockTest – GitHub link):

1. A mock PricingService is created using Mockito.
2. The behavior of calculatePriceQuantity() is stubbed to always return 100.00.
3. The mock is injected into ShoppingCartPricingServiceV2.
4. The system under test calls computeLineTotal().
5. The test verifies:
 - The returned value matches the mocked value.
 - calculatePriceQuantity() was invoked exactly once with the correct arguments.
 - An additional edge case verifies that when quantity ≤ 0 , the pricing service is not called.

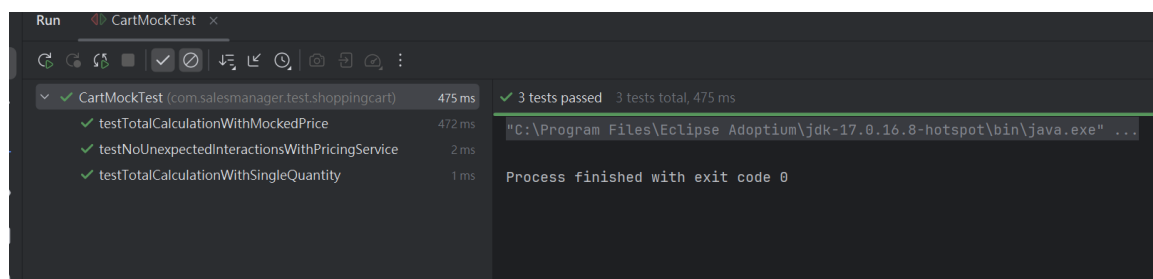
Why This Is Correct Mocking

Originally, the test directly invoked the mock object itself. That approach only demonstrated Mockito usage but did not test system behavior.

After refactoring, the mock is injected into the system under test. The test now verifies that:

- The shopping cart logic actually delegates pricing to PricingService.
- The correct parameters are passed.
- The pricing service is invoked exactly once.
- The cart correctly uses the returned value.

This transforms the test from “testing a mock object” into properly verifying the interaction between the system under test and its dependency, which is the core purpose of mocking.



```

@Test  👤 Yu Chieen
public void testTotalCalculationWithMockedPrice() {
    PricingService mockPricing = Mockito.mock(PricingService.class);

    when(mockPricing.calculatePriceQuantity(any(BigDecimal.class), eq(value: 1)))
        .thenReturn(new BigDecimal(val: "100.00"));

    ShoppingCartPricingServiceV2 sut = new ShoppingCartPricingServiceV2(mockPricing);

    BigDecimal result = sut.computeLineTotal(new BigDecimal(val: "50.0"), qty: 1);

    assertEquals(new BigDecimal(val: "100.00"), result);

    verify(mockPricing, times(wantedNumberOfInvocations: 1))
        .calculatePriceQuantity(new BigDecimal(val: "50.0"), quantity: 1);
}

@Test  new *
public void computeLineTotal_whenQtyIsZero_returnsZero_andDoesNotCallPricingService() {
    PricingService mockPricing = Mockito.mock(PricingService.class);

    ShoppingCartPricingServiceV2 sut = new ShoppingCartPricingServiceV2(mockPricing);

    BigDecimal result = sut.computeLineTotal(new BigDecimal(val: "50.0"), qty: 0);

    assertEquals(BigDecimal.ZERO, result);

    // Key point: no interaction with dependency when qty <= 0
    verifyNoInteractions(mockPricing);
}

```

-
- [CartPricingMockTest](#)

Conclusion

By refactoring the code to use interfaces and dependency injection, we transformed a legacy "untestable" component into a modular service. Using Mockito allowed us to verify business logic (like email generation) without the overhead or risk of sending real emails.

The combined approach of:

- Stubbing (for data isolation, e.g., IntegrationModuleSource)
- Mocking (for behavior verification, e.g., EmailService, PricingService)

...ensures our tests are fast, deterministic, isolated, and maintainable.